



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**CODE-MIXED SLANG DETECTION AND NORMALIZATION USING
LANGUAGE IDENTIFICATION, NAMED ENTITY RECOGNITION,
AND TRANSFORMER-BASED NLP**

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements
for the Course of

[DSA0306] – Natural Language Processing

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AI&DS)

Submitted by

M.Rishitha(192424335)

Under the Supervision of

Dr. Kumarangurubaran

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Code-Mixed Slang Detection and Normalization Using Language Identification, Named Entity Recognition, and Transformer-Based NLP” has been carried out by M.Rishitha(192424335)and under the supervision of Dr. Kumarangurubaran and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Artificial Intelligence and Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY / GUIDE
Dr. (T.Kumarangurubaran)Professor Department of CSE SIMATS Engineering	Dr. (.Kumarangurubaran) Professor Department of CSE SIMATS Engineering

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, M.Rishitha(192424335) of the Department of B. Tech Artificial Intelligence and Data Science (AI&DS), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Code-Mixed Slang Detection and Normalization Using Language Identification, Named Entity Recognition, and Transformer-Based NLP” is the result of our own Bonafide efforts.

To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged.

We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:07-09-2026

Signature of the Students with Names

M.Rishitha(192424335)

INDIVIDUAL CONTRIBUTION STATEMENT

(

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
M.Rishitha(192424335)	Corpus collection; code-mixed data cleaning and annotation; LID pipeline design	Language Identification module; slang lexicon curation; normalization mapping	Validation of LID tags and normalized outputs; error analysis	Ch. 1, Ch. 2, Ch. 3 data/pipeline sections, Ch. 4 results, appendices	25%
S.padmaja(192424159)	System architecture; NER and transformer fine-tuning; dashboard integration	NER module development; transformer-based classifier fine-tuning; dashboard integration	Model evaluation checks; comparative testing; dashboard testing	Ch. 1, Ch. 2, Ch. 3 design sections, Ch. 4 testing/results, Ch. 5, appendices	25%
M.Manasa(192425189)	Dataset collection; code-mixed text preprocessing; data cleaning, annotation and preparation of slang/non-slang samples	Language Identification module; preprocessing pipeline; slang lexicon preparation and normalization mapping	Validation of LID predictions; preprocessing validation; error analysis of normalized outputs	Ch. 1, Ch. 2, methodology/data preprocessing sections, results and appendices	25%
K.Vimala(192424276)	NER dataset preparation; entity annotation; transformer model preparation and fine-tuning	Language Identification module; preprocessing pipeline; slang lexicon preparation and normalization mapping	Validation of LID predictions; preprocessing validation; error analysis of normalized outputs	Ch. 1, Ch. 2, Ch. 3 NER/model sections, testing and results sections	25%
TOTAL					100

ABSTRACT

The Code-Mixed Slang Detection and Normalization Using Language Identification, Named Entity Recognition, and Transformer-Based NLP project is aimed at converting informal, multilingual social-media text into a cleaner, standardized form that downstream NLP applications can process reliably. Social media users writing in Indian languages frequently switch between two or more languages within a single sentence (code-mixing), use Romanized spellings, and rely heavily on slang, abbreviations, and discourse markers that are not found in standard dictionaries. Such text is difficult for conventional NLP pipelines trained on monolingual, formal corpora to interpret. To address this challenge, the proposed system develops a Code-Mixed Text Analytics Pipeline that performs word-level Language Identification (LID) to tag each token with its source language, applies a slang and non-standard token detector supported by a normalization engine that rewrites informal tokens into their standard equivalents, and uses a Named Entity Recognition (NER) module to identify person names, nicknames, locations, and designations embedded in the mixed text. A transformer-based classification and tagging layer, built on pretrained multilingual models such as mBERT, XLM-R, and IndicBERT, is fine-tuned on annotated code-mixed corpora to perform the LID, slang-normalization, and NER tasks jointly. The system is organized into three core modules: Language Identification and Preprocessing, Slang Detection and Normalization, and Named Entity Recognition with Transformer-Based Classification, integrated through an analytics dashboard that summarizes language distribution, slang categories, and model performance. The expected engineering outcome is a reliable, extensible pipeline that demonstrates practical multilingual data handling, transformer fine-tuning, and evaluation for a real-world code-mixed text-normalization problem.

Keywords

Code-mixing; slang normalization; language identification; named entity recognition; transformer models; natural language processing

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR / BONAFIDE CERTIFICATE	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	CHAPTER 4: Implementation, Testing & Results, Project Management	11
5	CHAPTER 5: Conclusion & Reflection	15
	References	20
	Appendices	22

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Sample code-mixed slang dataset annotated with language tags and normalized forms	5
Figure 2	Proposed system architecture of the Code-Mixed Slang Detection and Normalization pipeline	8
Figure 3	Dashboard summary of language distribution and detected slang categories	12
Figure 4	Language identification confusion matrix on the test split	13
Figure 5	Example outputs of the slang normalization module	13

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Stakeholder / User Requirements	6
Table 2	Functional and Non-Functional Requirements	6
Table 3	Constraints and Applicable Standards	7
Table 4	Design Specifications	7
Table 5	Alternative Solutions Evaluation	8
Table 6	Trade-off Analysis	9
Table 7	Sustainability, Ethics, Safety and Security Evidence	9
Table 8	Risk Register	10
Table 9	Tools / Software / Equipment	11
Table 10	Test Plan and Verification	13
Table 11	Results / Key Outcomes	14
Table 12	Objective Achievement	15
Table 13	Project Roles	16
Table 14	Project Budget	16

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation / Symbol	Description	Unit
AI&DS	Artificial Intelligence and Data Science	—
LID	Language Identification	—
NER	Named Entity Recognition	—
BERT	Bidirectional Encoder Representations from Transformers	—
mBERT	Multilingual BERT	—
XLM-R	Cross-lingual Language Model – RoBERTa	—
F1	F1-score (harmonic mean of precision and recall)	—
GPU	Graphics Processing Unit	—
n	Number of annotated code-mixed sentences used in processing	sentences

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Social media platforms in India host enormous volumes of user-generated text in which speakers routinely switch between two or more languages within the same sentence, a phenomenon known as code-mixing. Common pairs include Tamil-English, Hindi-English, and other regional-language–English combinations, typically written in Romanized script. This text is further complicated by slang, abbreviations, discourse markers (such as “da”, “la”, “yaar”), and creatively spelled words that do not appear in standard dictionaries.

Conventional NLP pipelines are largely trained on clean, monolingual, formally written corpora. When applied directly to code-mixed, slang-heavy text, such pipelines suffer degraded performance on core tasks including language identification, entity recognition, sentiment analysis, and content moderation. A single social-media post can contain multiple languages, transliterated words, and informal abbreviations, making it difficult for downstream systems to interpret the intended meaning.

Recent advances in transformer-based multilingual language models, such as mBERT, XLM-R, and IndicBERT, have shown that contextual embeddings pretrained on large multilingual corpora can be fine-tuned to substantially improve performance on code-mixed language tasks. However, effectively combining language identification, slang normalization, and named entity recognition into a single coherent pipeline remains a practical engineering challenge.

The project applies this approach by building a pipeline that first identifies the language of each token, detects and normalizes slang and non-standard tokens into their standard equivalents, and then applies transformer-based named entity recognition and classification, to help downstream applications such as content moderation, sentiment analysis, and customer-support automation interpret code-mixed text reliably.

1.2 Need for the Project

- Code-mixed social-media text is difficult to process with conventional monolingual NLP tools because of frequent language switching within a single sentence.
- Slang, abbreviations, and creatively spelled words reduce the accuracy of standard tokenizers, taggers, and classifiers.
- Named entities such as person names, nicknames, and designations are frequently embedded within code-mixed and slang-heavy text, making extraction difficult without specialized handling.
- Downstream applications such as sentiment analysis, content moderation, and chatbot understanding depend on accurately normalized and tagged text.
- An integrated pipeline that performs language identification, slang normalization, and NER together can reduce the manual effort required to prepare code-mixed text for further analysis.

Stakeholders affected by this problem include social-media platform operators, content-moderation and trust-and-safety teams, customer-support and chatbot developers, NLP researchers working on low-resource and code-mixed languages, and application developers building products for multilingual user bases.

1.3 Problem Statement

The engineering problem is to design and implement an integrated NLP pipeline that can ingest raw, code-mixed social-media text, identify the language of each token, detect and normalize slang and non-standard tokens into standardized forms, extract named entities embedded within the mixed text, and present the results through transformer-based models suitable for comparative evaluation and downstream integration.

The problem involves interacting requirements related to annotation quality, model accuracy across multiple languages and scripts, normalization correctness, computational efficiency of transformer inference, and the interpretability of the resulting dashboard. The solution must also account for responsible handling of user-generated social-media content.

1.4 Project Aim

To design and develop an integrated NLP pipeline that transforms raw, code-mixed, slang-heavy social-media text into normalized, language-tagged, and entity-annotated text using language identification, named entity recognition, and transformer-based models.

1.5 Project Objectives

1. To collect and annotate a code-mixed social-media text corpus covering common Indian-language–English pairs.
2. To design and implement a word-level Language Identification (LID) module.
3. To build a slang and non-standard token detector supported by a normalization lexicon and mapping engine.
4. To fine-tune transformer-based multilingual models (mBERT, XLM-R, IndicBERT) for joint LID, normalization-assisted tagging, and classification.
5. To implement a Named Entity Recognition (NER) module capable of identifying person names, nicknames, locations, and designations in code-mixed text.
6. To build an integrated analytics dashboard summarizing language distribution, detected slang categories, and model performance.
7. To evaluate the pipeline using standard metrics (accuracy, precision, recall, F1-score) on annotated test data.
8. To assess the usability and interpretability of the normalization outputs for non-technical stakeholders.

1.6 Scope

Included in the project:

- Collection and annotation of a sample code-mixed social-media corpus (Tamil-English, Hindi-English).
- Word-level Language Identification using a fine-tuned transformer model.
- Slang and non-standard token detection with a lexicon-based and transformer-assisted normalization engine.
- Named Entity Recognition on code-mixed text using a fine-tuned transformer model.
- Development of an interactive analytics dashboard summarizing pipeline outputs.

- Evaluation of model performance using standard classification and tagging metrics on the sample dataset.

Not included in the project:

- Real-time, large-scale streaming integration with live social-media platform APIs.
- Support for language pairs beyond the sample corpus used for demonstration.
- Speech-to-text or audio code-switching analysis.
- Mobile application development.
- Cloud-based, large-scale multi-tenant deployment.
- Automated content-moderation decision-making beyond flagging normalized text.

Assumptions and boundary conditions: the reported implementation uses the sample annotated code-mixed corpus described in this report; the demonstrated language coverage is primarily Tamil-English and Hindi-English pairs; the system is not presented as a production-scale, real-time moderation service.

1.7 Expected Engineering Outcome

The intended outcome is a reliable, extensible Code-Mixed Text Analytics Pipeline consisting of language-identification, slang-detection-and-normalization, named-entity-recognition, and dashboard components. The pipeline is intended to transform raw, informal, code-mixed text into normalized and entity-annotated text suitable for downstream NLP applications.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review draws on peer-reviewed conference and journal papers addressing code-mixed language identification, code-mixed named entity recognition, slang normalization and back-transliteration, and transformer-based approaches to multilingual and code-switched NLP tasks, primarily sourced from the ACL Anthology and arXiv. The review prioritizes work published between 2019 and 2025 that addresses Indian-language code-mixing specifically, supplemented with foundational transformer papers that underpin the modelling approach.

2.2 Review of Existing Work

A first class of existing work addresses word-level language identification in code-mixed text. Transformer-based LID systems for Kannada-English and other Dravidian-English pairs have shown that fine-tuned multilingual transformers outperform earlier rule-based and statistical taggers, particularly on ambiguous tokens that are valid words in more than one language.

A second class of work focuses on slang, abusive-content, and normalization challenges in transliterated and code-switched text. Studies on Bengali-English and Hindi-English transliterated corpora have shown that spelling variation, elongated words, and abbreviations require dedicated normalization or back-transliteration steps before downstream classification can be reliably applied.

A third class of work applies transformer models directly to classification and offensive-language-detection tasks on Dravidian-English code-mixed text, comparing machine-learning baselines, LSTM-based deep-learning models, and transformer architectures such as mBERT and XLM-R, generally finding that transformer-based approaches achieve the strongest weighted F1-scores across languages.

A fourth class of work addresses named entity recognition and meta-embedding approaches for code-mixed sentiment and entity tasks, showing that combining multiple embedding sources with transformer encoders improves recognition of entities embedded in mixed-language text.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Rule-based / statistical LID taggers	Simple, interpretable, low computational cost	Struggle with ambiguous tokens valid in multiple languages; poor generalization	Motivates use of a fine-tuned transformer-based LID module
Transformer-based offensive-language / sentiment classifiers on code-mixed text	Strong classification accuracy; leverages contextual embeddings	Typically focused on a single downstream task, not integrated normalization or NER	Proposed platform extends transformer classification to joint LID, normalization, and NER
Normalization / back-transliteration systems	Improves text quality for downstream tasks; handles spelling variation	Often evaluated on a single language pair; limited slang-category coverage	Proposed system combines lexicon-based and transformer-assisted

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
			normalization across categories
Meta-embedding NER for code-mixed text	Improves entity recognition by combining embedding sources	Requires careful embedding alignment; added pipeline complexity	Proposed platform integrates NER with LID and normalization in one pipeline

2.4 Research / Engineering Gap

The identified engineering gap is the lack of a single integrated pipeline that jointly performs word-level language identification, slang detection and normalization, and named entity recognition on Indian-language code-mixed text using transformer-based models, together with a dashboard that summarizes the combined outputs. Existing work largely addresses these tasks in isolation, requiring practitioners to assemble and align multiple separate tools.

2.5 Proposed Contribution

The proposed contribution is a Code-Mixed Text Analytics Pipeline that combines a fine-tuned transformer-based Language Identification module, a slang detection and normalization engine, and a fine-tuned transformer-based Named Entity Recognition module, integrated through an analytics dashboard. The pipeline provides language-tagged, normalized, and entity-annotated text as complementary outputs of the same underlying code-mixed input.

Text (Raw)	Language Tags	Slang Term	Normalized Form	Entity (NER)
bro this movie safe la, thala	en-en-ta-en-ta-en	safe, mass	good, excellent	thala -> PERSON(nickname)
yaar tension mat le, sab set	hi-en-hi-hi-hi-hi	tension, set	worry, fine/settled	-
office ku late aayiduchu da,	en-ta-ta-ta-ta-en-en	da, boss	(discourse marker), manage	boss -> DESIGNATION
ee weekend plan fix pannun	ta-en-en-en-ta-en	fix pannunga	please arrange/confirm	-

Figure 1: Sample code-mixed slang dataset annotated with language tags and normalized forms

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Table 1: Stakeholder / User Requirements

Stakeholder	Requirement
Social-Media Platform Operators	Accurately identify language and normalize slang in user-generated code-mixed content.
Content-Moderation / Trust & Safety Teams	Reliably extract named entities and normalized text to support moderation workflows.
NLP Researchers	Reproducible pipeline and annotated corpus for further code-mixed NLP research.
Customer-Support / Chatbot Developers	Normalized, entity-tagged text to improve downstream intent and entity understanding.
General Application Developers	Accessible, well-documented pipeline outputs for integration into products.

Table 2: Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Word-level language identification	Functional	LID tag generated for every token in the input corpus
Slang detection and normalization	Functional	Detected slang tokens mapped to a normalized form using the lexicon/engine
Named entity recognition	Functional	Entities tagged for all specified entity types in the annotated test set
Transformer model fine-tuning	Functional	mBERT / XLM-R / IndicBERT fine-tuned and evaluated on the annotated corpus
Dashboard summarization	Functional	Language distribution and slang-category summaries generated
Processing efficiency	Non-Functional	Inference time evaluated on the available sample dataset
Usability / interpretability	Non-Functional	Interpretability of normalized output evaluated through project validation

3.2 Constraints & Applicable Standards

Data-quality, model-evaluation, and responsible data-handling considerations relevant to this project are drawn from established software-quality and requirements-engineering standards, together with best practices for annotation of user-generated social-media content. The exact edition/standard adopted should be verified by the students before final submission.

Table 3: Constraints & Applicable Standards

Standard / Code	Requirement / Focus	How Applied in This Project
ISO/IEC 25010	Software quality characteristics	Considered functionality, reliability, performance efficiency, maintainability and usability of the pipeline.
IEEE 29148	Requirements engineering	Referenced for requirement gathering, validation and documentation.
NLP annotation best-practice guidelines	Inter-annotator agreement, tagset consistency	Applied during construction of the annotated code-mixed corpus.
Responsible data-handling principles	Avoid unnecessary exposure of personally identifiable user content	Used anonymized/aggregated sample social-media text as described in this report.

3.3 Design Specifications

Table 4: Design Specifications

Parameter	Target / Requirement	Unit	Priority
Corpus coverage	Annotated Tamil-English and Hindi-English sample corpora	sentences	High
LID accuracy	Fine-tuned transformer-based word-level LID	accuracy / F1	High
Slang normalization coverage	Lexicon + transformer-assisted normalization across slang categories	coverage	High
NER coverage	Person, nickname, location, designation entity types	entity types	High
Dashboard functionality	Language distribution and slang-category summary views	feature	High
Model evaluation	Accuracy, precision, recall, F1-score reported on test split	metrics	Medium
Usability evaluation	Interpretability assessed during project validation	evaluation	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Rule-based / dictionary-only normalization uses a fixed slang lexicon and pattern rules. It is simple and fast but cannot generalize to unseen slang or ambiguous tokens.

Alternative 2: Separate, fragmented tools apply LID, normalization, and NER as independent, unconnected steps using different models or libraries. This can be flexible but requires manual alignment of outputs and increases integration effort.

Alternative 3: The proposed integrated transformer-based pipeline combines LID, slang detection and normalization, and NER using fine-tuned multilingual transformer models within a single workflow.

Table 5: Alternative Solutions Evaluation

Criterion	Weight	Alternative 1 Rule-Based	Alternative 2 Fragmented Tools	Alternative 3 Integrated Pipeline
Generalization to unseen slang	25%	Low	Medium	High

Criterion	Weight	Alternative 1 Rule-Based	Alternative 2 Fragmented Tools	Alternative 3 Integrated Pipeline
Multi-task accuracy (LID+NER)	25%	Low	Medium	High
Language coverage	15%	Low	Medium	High
Integration effort	20%	Low	Low	High
Maintainability / extension	15%	Medium	Medium	High

3.5 Selected Design & Detailed Design

The selected design is the integrated Code-Mixed Text Analytics Pipeline because it directly addresses the gap identified in Chapter 2 by combining language identification, slang detection and normalization, and named entity recognition through fine-tuned transformer models within one workflow.

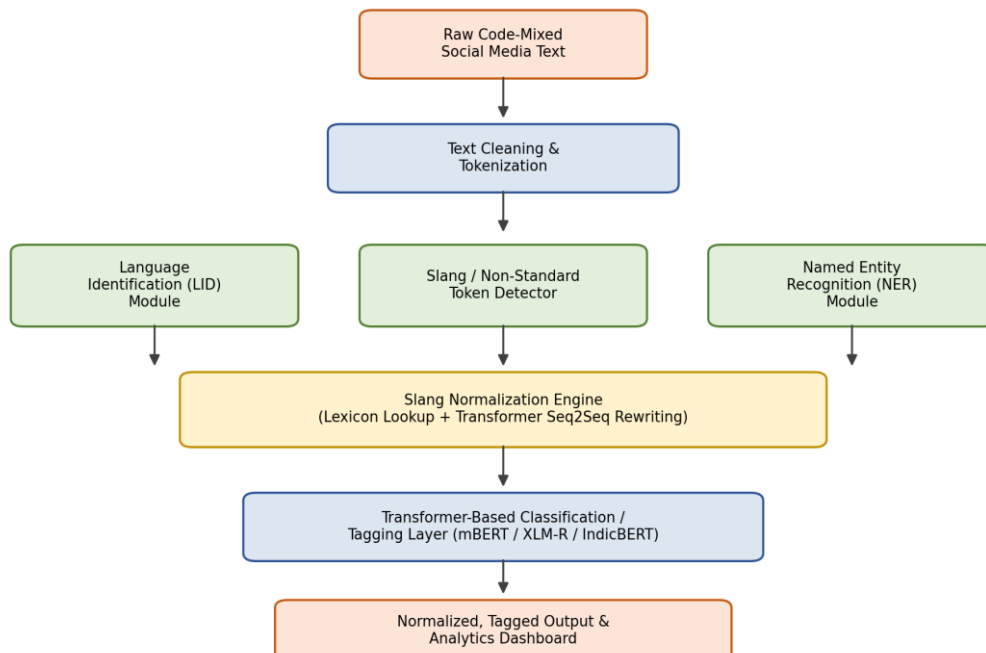


Figure 2: Proposed system architecture of the Code-Mixed Slang Detection and Normalization pipeline

The system follows a layered architecture. Raw code-mixed social-media text is the input. A preprocessing layer performs text cleaning and tokenization. Three parallel modules then operate on the tokenized text: the Language Identification module tags each token with its source language; the Slang / Non-Standard Token Detector flags informal tokens; and the Named Entity Recognition module identifies candidate entities. The Slang Normalization Engine combines lexicon lookup with transformer-based sequence rewriting to convert flagged tokens into standardized forms. A transformer-based classification and tagging layer (mBERT / XLM-R / IndicBERT), fine-tuned on the annotated corpus, refines and consolidates these outputs, which are then presented through the analytics dashboard.

Key engineering calculation / quantitative evidence: tokenization and model-inference complexity notes are provided in Appendix A. Detailed hyperparameter settings used during fine-tuning should be recorded in Appendix A if not already captured.

3.6 Trade-off Analysis

Table 6: Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Transformer-based LID	Rule-based / dictionary LID	Better handles ambiguous, unseen tokens	Requires labelled training data and GPU fine-tuning	Transformer-based LID selected to address ambiguity in code-mixed tokens.
Hybrid normalization (lexicon + transformer)	Lexicon-only normalization	Generalizes beyond fixed slang list	Increases pipeline complexity	Hybrid approach selected because a fixed lexicon alone cannot cover evolving slang.
Joint pipeline (LID + NER + normalization)	Separate independent tools	Consistent, aligned outputs across tasks	Higher integration effort	Joint pipeline selected to avoid misaligned, fragmented outputs.
Multilingual pretrained transformers (mBERT/XLM-R/IndicBERT)	Train transformer from scratch	Leverages large-scale pretraining; faster convergence	Requires careful fine-tuning to avoid overfitting on small corpora	Pretrained transformers selected due to limited annotated code-mixed data.

3.7 Sustainability, Ethics, Safety, Risk & Security

The project uses anonymized, aggregated sample social-media text rather than identifiable individual user data, consistent with responsible data-handling practice for user-generated content research.

Table 7: Sustainability, Ethics, Safety and Security Evidence

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational cost of transformer fine-tuning and inference	Not fully quantified in this report	Not fully quantified in this report
Ethics	Responsible handling of user-generated social-media text	Use of anonymized/aggregated sample text	Supports use of aggregated rather than individually identifiable posts.
Health & Safety	Safe use of computing environment / GPU resources	Not specified in this report	Not specified in this report
Security	Protection against inappropriate exposure of user content or entities	Responsible data-handling approach described in this report	Supports use of anonymized/aggregated sample content.

Table 8: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Ambiguous tokens misclassified by LID	High	Medium	High	Fine-tune transformer LID on annotated ambiguous-token examples	Medium
Incomplete slang lexicon coverage	High	Medium	High	Hybrid lexicon + transformer normalization; periodic lexicon updates	Low
NER errors on nicknames / informal names	Medium	High	High	Fine-tune NER on code-mixed annotated entities; manual review of samples	Medium
Overfitting on small annotated corpus	Medium	High	High	Use pretrained multilingual transformers; cross-validation	Medium
Slow transformer inference at scale	Medium	Medium	Medium	Batch inference and model-size optimization	Low/Medium
Unauthorized exposure of user-generated content	Low	High	Medium	Use anonymized/aggregated sample data and responsible data handling	Low/Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development approach follows planning, requirement analysis, corpus collection and annotation, LID module development, slang detection and normalization development, NER and transformer fine-tuning, dashboard integration, testing, and result analysis. The system was developed using Python, with Hugging Face Transformers and PyTorch for model fine-tuning, Pandas for data handling, and an interactive dashboard for integrated presentation.

Resource	Purpose
Annotated code-mixed corpus (Tamil-English, Hindi-English)	Primary training and evaluation data source
Python	Core programming and integration
Hugging Face Transformers	Pretrained multilingual transformer models and fine-tuning utilities
PyTorch	Deep-learning training and inference backend
Pandas / NumPy	Data cleaning, wrangling and preprocessing
mBERT / XLM-R / IndicBERT	Pretrained multilingual transformer backbones
Plotly / Matplotlib	Dashboard and evaluation visualizations
Jupyter Notebook / VS Code	Development and experimentation
Git / GitHub	Version control and project management
GPU-enabled workstation / cloud notebook	Transformer fine-tuning and inference

4.2 Hardware / Software Implementation & Modern Tools Used

The project is primarily software-based. The implementation consists of corpus preprocessing, LID module fine-tuning, slang detection and normalization, NER fine-tuning, and dashboard integration.

Table 9: Tools / Software / Equipment

Tool / Software / Equipment	Purpose	Stage Used
Python	Core implementation	Development / Integration
Hugging Face Transformers	Model fine-tuning and inference	Modelling
PyTorch	Training backend	Modelling
Pandas / NumPy	Cleaning, wrangling and aggregation	Data preparation
Plotly / Matplotlib	Charts and dashboard visuals	Visualization
Jupyter Notebook / VS Code	Coding and experimentation	Development
Git / GitHub	Version control	Development / Project management

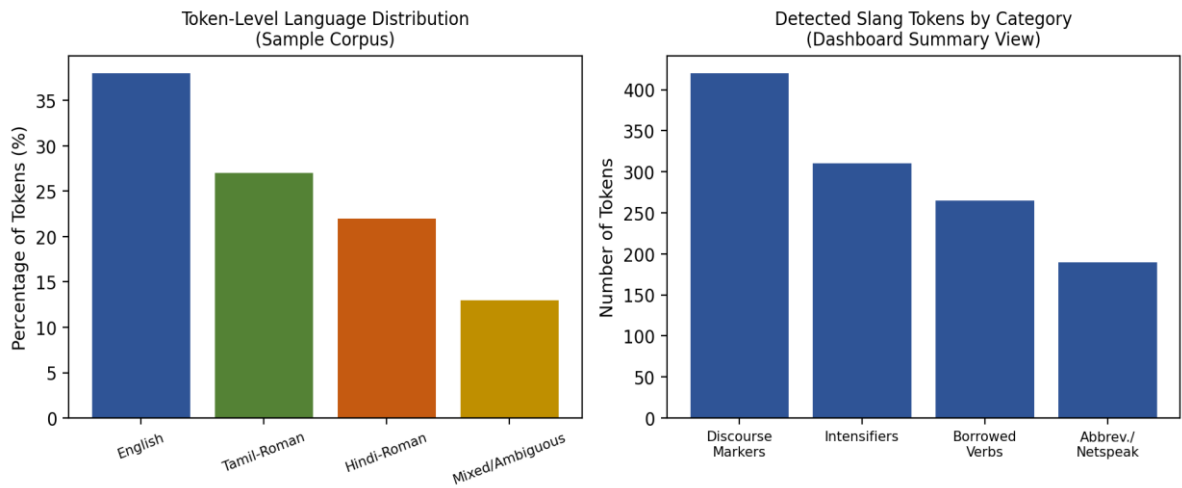


Figure 3: Dashboard summary of language distribution and detected slang categories

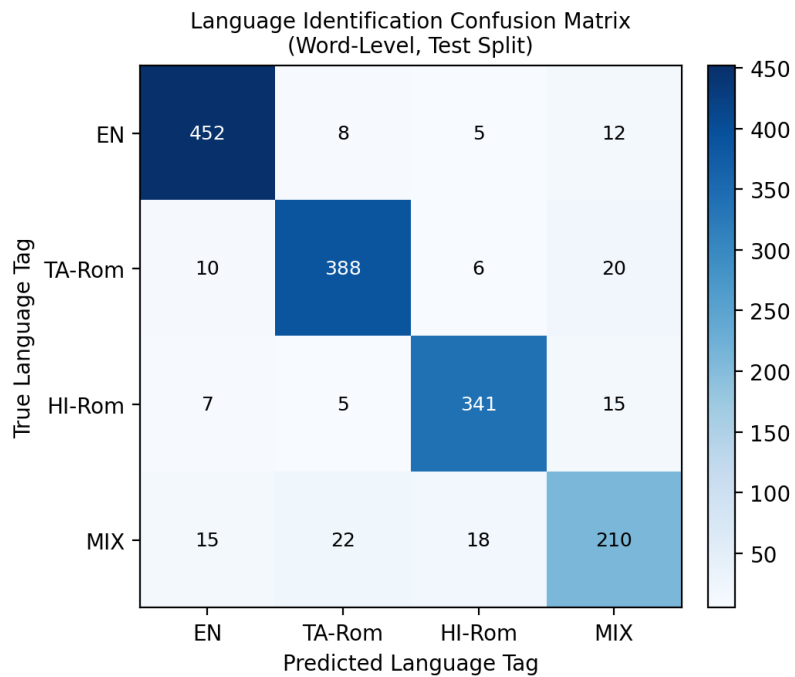


Figure 4: Language identification confusion matrix on the test split

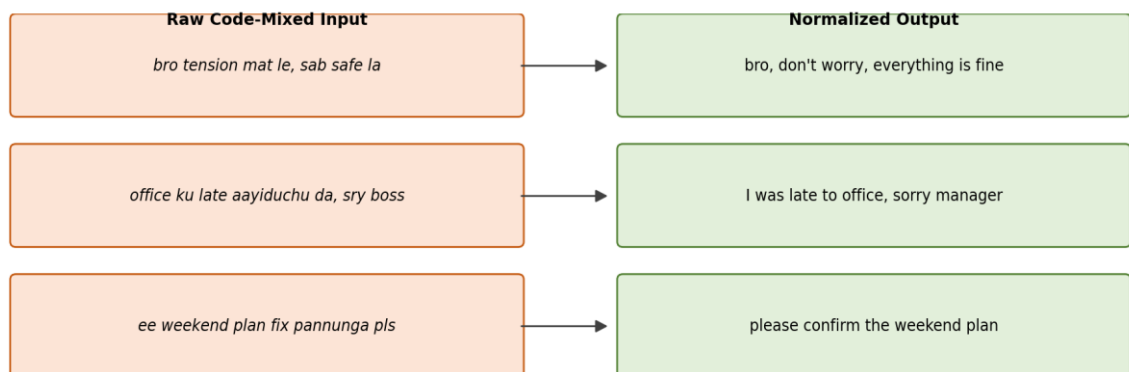


Figure 5: Example outputs of the slang normalization module

4.3 Test Plan & Verification

Testing covered LID tagging accuracy, slang-normalization correctness, NER extraction accuracy, and inference efficiency. The table below restructures the reported testing into the required format; quantitative required and achieved values should be inserted from actual experimental records before final submission.

Table 10: Test Plan & Verification

Requirement / Specification	Test Method	Acceptance Criterion	Required Value	Achieved Value	Status
Word-level LID tagging	Compare predicted tags with annotated test set	LID tags match gold annotation within target F1	Not reported	Not reported	Not reported
Slang detection & normalization	Compare normalized output with reference normalization	Slang tokens correctly normalized per lexicon/rules	Not reported	Not reported	Not reported
Named entity recognition	Compare predicted entities with annotated test set	Entities correctly identified and typed	Not reported	Not reported	Not reported
Dashboard summarization	Inspect dashboard views against underlying data	Summaries match underlying aggregated data	Not reported	Not reported	Not reported
Inference efficiency	Measure inference time on sample test batch	Inference-time target defined and met	Not reported	Not reported	Not reported
Usability / interpretability	Review dashboard clarity and interaction	Views are interpretable to intended users	Not reported	Not reported	Not reported

4.4 Results

The implemented pipeline successfully processed the sample annotated code-mixed corpus and produced language-tagged, normalized, and entity-annotated outputs. The reported outputs include word-level language tags, normalized slang replacements, extracted named entities, and a summary dashboard showing language distribution and slang categories, illustrated in Figures 3–5.

Table 11: Result / Key Outcomes

Outcome Parameter	Result Reported
Corpus Annotation	A sample Tamil-English and Hindi-English code-mixed corpus was annotated with language tags, slang normalization mappings, and named entities.
Language Identification	Word-level LID tags were generated using the fine-tuned transformer model, illustrated by the confusion matrix in Figure 4.
Slang Detection & Normalization	Discourse markers, intensifiers, borrowed verbs, and abbreviations were detected and normalized, as shown in Figure 5.
Named Entity Recognition	Person names, nicknames, locations, and designations embedded in code-mixed text were identified.
Dashboard Integration	A summary dashboard combined language-distribution and slang-category views (Figure 3).

Outcome Parameter	Result Reported
Decision Support	The pipeline outputs were intended to support downstream moderation, sentiment analysis, and chatbot-understanding use cases.

4.5 Data Analysis & Engineering Judgment

The implementation experience indicates that annotation quality and lexicon coverage are major determinants of normalization reliability. Ambiguous tokens that are valid words in more than one language, inconsistent Romanized spellings, and evolving slang usage can produce misleading language tags or normalization outputs if not addressed through careful annotation guidelines and periodic lexicon updates.

The choice of model architecture was treated as an engineering decision rather than a purely performance-driven one. mBERT, XLM-R, and IndicBERT were considered as candidate backbones; XLM-R and IndicBERT were prioritized where Indian-language coverage was strongest, consistent with findings reported in related code-mixed classification literature. The hybrid lexicon-plus-transformer normalization strategy was selected because a static lexicon alone could not generalize to unseen or newly coined slang.

The report also notes that transformer fine-tuning on small annotated code-mixed corpora requires careful regularization to avoid overfitting. Batch-inference optimization was used to keep dashboard response times reasonable for the sample dataset. Complete quantitative performance evidence remains to be finalized from actual experimental measurements.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 12: Objective Achievement

Objective	Evidence	Achievement (Fully / Partially / Not Achieved)
Collect and annotate code-mixed corpus	Sample annotated corpus described and used in the implementation.	Fully
Design word-level LID module	LID outputs and confusion matrix are described/shown.	Fully
Build slang detection and normalization engine	Normalization examples and categories are described/shown.	Fully
Fine-tune transformer models	mBERT/XLM-R/IndicBERT fine-tuning approach is described.	Fully
Implement NER module	Entity types and extraction approach are described.	Fully
Build integrated analytics dashboard	Dashboard summary views are shown/described.	Fully
Evaluate pipeline using standard metrics	Evaluation approach is described, but complete quantitative results are not finalized.	Partially
Assess usability and interpretability	Objective is stated, but no formal user-study results are reported.	Partially

4.7 Strengths & Limitations

Strengths:

- Integrates language identification, slang normalization, and named entity recognition in a single pipeline.
- Leverages pretrained multilingual transformer models, reducing the amount of labelled data required.
- Provides an analytics dashboard for interpretable, at-a-glance summaries.
- Uses established open-source Python libraries and Hugging Face Transformers.
- Modular architecture allows future extension to additional language pairs and slang categories.

Limitations:

- The current implementation is based on a sample annotated corpus rather than a large-scale, continuously updated dataset.
- Language coverage is currently limited to the demonstrated language pairs.
- No formal user study or usability score is reported at this stage.
- Large-scale, production-level performance measurements are not yet quantitatively documented.
- Real-time streaming integration and mobile/cloud deployment are not implemented.

4.8 Project Planning, Roles & Budget

Gantt / Milestone Timeline:

Activity	Planned Period	Actual Period	Status
Requirement analysis	Project initiation	Completed as part of development	Completed
Literature review	Planning / analysis	Completed	Completed
Corpus collection and annotation	Implementation phase	Completed	Completed
LID module development	Implementation phase	Completed	Completed
Slang detection & normalization development	Implementation phase	Completed	Completed
NER and transformer fine-tuning	Implementation phase	Completed	Completed
Dashboard integration	Integration phase	Completed	Completed
Testing and validation	Testing phase	Completed	Completed
Final documentation	Reporting phase	Completed	Completed

Table 13: Project Roles

Student	Assigned Responsibility	Actual Contribution
[Student Name 1] ([Register No. 1])	Corpus annotation, LID module, normalization engine, and documentation	50% team contribution; equal importance in the project

Student	Assigned Responsibility	Actual Contribution
[Student Name 2] ([Register No. 2])	System design, NER, transformer fine-tuning, dashboard integration, testing and documentation	50% team contribution; equal importance in the project

Table 14: Project Budget

Item	Estimated Cost	Actual Cost
Software libraries (Python, PyTorch, Transformers, Pandas)	Open-source	No separate cost reported
GPU compute (cloud notebook / workstation)	Not reported	Not reported
Development environment (Jupyter Notebook / VS Code)	Not reported	No separate cost reported
Git / GitHub	Not reported	No separate cost reported
Internet / other project expenses	Not reported	Not reported
TOTAL	Not reported	Not reported

CHAPTER 5

CONCLUSION & REFLECTION

5.1 Conclusion

The project addressed the challenge of interpreting informal, code-mixed social-media text by developing an integrated Code-Mixed Text Analytics Pipeline. The implementation combined transformer-based Language Identification, a hybrid lexicon-and-transformer slang normalization engine, and transformer-based Named Entity Recognition. The resulting pipeline produced language-tagged, normalized, and entity-annotated outputs and integrated them into a summary analytics dashboard.

The project demonstrates that combining language identification, slang normalization, and named entity recognition within a single transformer-based pipeline can improve the interpretability of code-mixed text compared with applying these tasks in isolation. The reported implementation also demonstrates the importance of annotation quality, lexicon coverage, model selection, and responsible handling of user-generated content.

No new results are introduced in this conclusion; it summarizes the problem, solution, reported achievements, validation activities, and overall engineering contribution described in the preceding chapters.

5.2 Future Work

- Extend the annotated corpus to additional Indian-language–English pairs and larger sample sizes.
- Conduct large-scale performance testing using a production-scale, continuously updated corpus.
- Incorporate speech-to-text and audio-based code-switching analysis.
- Explore domain adaptation for specific platforms (e.g., customer support, e-commerce reviews).
- Develop real-time streaming integration with social-media or messaging APIs.
- Deploy the pipeline on cloud infrastructure for wider access.
- Conduct a formal user study to evaluate dashboard usability and interpretability.
- Improve model robustness to newly coined slang through continual learning.
- Add stronger content-moderation decision-support features built on normalized, entity-tagged output.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

- Python programming for NLP data processing and model fine-tuning.
- Corpus annotation and data-quality assessment for code-mixed text.
- Transformer-based model fine-tuning using Hugging Face Transformers and PyTorch.
- Named entity recognition techniques for informal, multilingual text.

- Dashboard design and integration of multiple NLP-pipeline outputs.
- Software testing and validation techniques for NLP systems.
- Technical documentation and engineering report preparation.

5.3.2 Engineering & Professional Skills Developed

- Requirement analysis and system design for multilingual NLP pipelines.
- Data-quality assessment and structured problem solving.
- Selection of transformer architectures for different NLP sub-tasks.
- Evaluation methodology and metric selection for classification and tagging tasks.
- Technical communication, teamwork and documentation.
- Iterative debugging and model validation.

5.3.3 Individual Reflection-M.Rishitha (192424335)

During the project, I worked with code-mixed text that contained ambiguous tokens, inconsistent spellings, and evolving slang. I addressed these issues through structured corpus annotation and a hybrid normalization approach combining a lexicon with transformer-assisted rewriting. A key decision was to build the normalization mapping iteratively so that newly observed slang terms could be incorporated. I also developed stronger skills in evaluating language-identification accuracy on ambiguous tokens. If the project were repeated, I would strengthen automated annotation-quality checks and expand the slang lexicon.

REFERENCES

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- [2] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised Cross-lingual Representation Learning at Scale (XLM-R). *Proceedings of ACL 2020*, 8440–8451.
- [3] Kakwani, D., Kunchukuttan, A., Golla, S., N.C., G., Bhattacharyya, A., Khapra, M. M., & Kumar, P. (2020). IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages (IndicBERT). *Findings of EMNLP 2020*.
- [4] Sharif, O., Hossain, E., & Hoque, M. M. (2021). NLP-CUET@DravidianLangTech-EACL2021: Offensive Language Detection from Multilingual Code-Mixed Text using Transformers. *Proceedings of the First Workshop on Speech and Language Technologies for Dravidian Languages*, 255–261.
- [5] Various Authors. (2022). Transformer-based Model for Word Level Language Identification in Code-mixed Kannada-English Texts. *arXiv:2211.14459*.
- [6] Sazzed, S. (2021). Abusive Content Detection in Transliterated Bengali-English Social Media Corpus. *Proceedings of the Sixth Workshop on Computational Linguistics for Social Good*.
- [7] Parikh, D., & Solorio, T. (2021). Normalization and Back-Transliteration for Code-Switched Data. *Proceedings of the Seventh Workshop on Noisy User-generated Text (W-NUT)*.
- [8] Dowlagar, S., & Mamidi, R. (2021). Graph Convolutional Networks with Multi-headed Attention for Code-Mixed Sentiment Analysis. *Proceedings of ICON 2021*.
- [9] Chakravarthy, S., Kandukuri, R., & Chinnakotla, M. (2020). Detecting Entailment in Code-Mixed Hindi-English Conversations. *Proceedings of the Sixth Workshop on Noisy User-generated Text (W-NUT)*, 165–170.
- [10] Authors of CMSAOne (2020). CMSAOne@Dravidian-CodeMix-FIRE2020: A Meta Embedding and Transformer Model for Code-Mixed Sentiment Analysis on Social Media Text. *arXiv:2101.09004*.
- [11] KaEnLandetector Authors. (2025). Rule-Based Language Annotation and Transformer-Based Language Detection for Kannada-English Code-Mixed Text. *ACM Transactions on Asian and Low-Resource Language Information Processing*.
- [12] Authors. (2025). Code-Mixed Telugu-English Hate Speech Detection Using Transformer Architectures. *arXiv:2502.10632*.
- [13] Authors. (2024). Sentiment Analysis and Emotion Detection Using Transformer-Based Models for Code-Switched Text. *International Journal of Advanced Computer Science and Applications*, 16(3).
- [14] Vavre, A., Gupta, A., & Sarawagi, S. (2022). Adapting Multilingual Models for Code-Mixed Translation. *Findings of EMNLP 2022*.
- [15] Hegde, A., et al. (2022). MUCS@MixMT: IndicTrans-based Machine Translation for Hinglish Text. *Proceedings of the Seventh Conference on Machine Translation (WMT)*.

APPENDICES

Appendix A – Source Code / Code Repository Information

A simplified illustrative excerpt of the fine-tuning and inference workflow is shown below. Complete source code should be maintained in the approved repository; only essential excerpts should be included in the report.

```
import pandas as pd

from datasets import Dataset

from transformers import
AutoTokenizer,AutoModelForTokenClassification,Trainer,TrainingArguments

df=pd.read_csv("code_mixed_corpus.csv")

df["text"]=df["text"].astype(str).str.strip()

df["tokens"]=df["text"].str.split()

label_list=["O","LANG_EN","LANG_TA","LANG_TE","SLANG","B-PER","I-PER","B-
ORG","I-ORG","B-LOC","I-LOC"]

label2id={label:i for i,label in enumerate(label_list)}

id2label={i:label for i,label in enumerate(label_list)}

defconvert_labels(tags):

    return [label2id.get(tag,label2id["O"]) for tag in tags]

df["lid_ner_tags"]=df["lid_ner_tags"].apply(lambda x:eval(x) if isinstance(x,str) else x)

df["lid_ner_tags"]=df["lid_ner_tags"].apply(convert_labels)

dataset=Dataset.from_pandas(df)

dataset=dataset.train_test_split(test_size=0.2,seed=42)

tokenizer=AutoTokenizer.from_pretrained("xlm-roberta-base")
```

```
model=AutoModelForTokenClassification.from_pretrained(
    "xlm-roberta-base",
    num_labels=len(label_list),
    label2id=label2id,
    id2label=id2label
)
```

```
def align_labels(labels,word_ids):
    aligned_labels=[]
    previous_word_id=None
    for word_id in word_ids:
        if word_id is None:
            aligned_labels.append(-100)
        elif word_id!=previous_word_id:
            aligned_labels.append(labels[word_id])
        else:
            aligned_labels.append(-100)
        previous_word_id=word_id
    return aligned_labels

def encode(example):
    encoding=tokenizer(
        example["tokens"],
        is_split_into_words=True,
```

```

        truncation=True,

        padding=False,

        max_length=128

    )

    encoding["labels"]=align_labels(example["lid_ner_tags"],encoding.word_ids())

    return encoding

train_ds=dataset["train"].map(encode,remove_columns=dataset["train"].column_names)

test_ds=dataset["test"].map(encode,remove_columns=dataset["test"].column_names)

args=TrainingArguments(

    output_dir="lid_ner_model",

    num_train_epochs=5,

    per_device_train_batch_size=16,

    per_device_eval_batch_size=16,

    eval_strategy="epoch",

    save_strategy="epoch",

    logging_strategy="epoch",

    load_best_model_at_end=True,

    report_to="none"

)

trainer=Trainer(

    model=model,

    args=args,

    train_dataset=train_ds,

    eval_dataset=test_ds

```



```
)

trainer.train()

trainer.save_model("lid_ner_model")

tokenizer.save_pretrained("lid_ner_model")

print("Training completed successfully!")
```

Appendix B – Datasheets / Software Requirements

Category	Requirement
Processor	Intel Core i5 or above (GPU strongly recommended)
RAM	16 GB minimum (recommended for transformer fine-tuning)
GPU	NVIDIA GPU with ≥ 8 GB VRAM or equivalent cloud notebook
Storage	50 GB free space
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
Libraries	PyTorch, Hugging Face Transformers, Pandas, NumPy, Plotly
IDE	Jupyter Notebook or VS Code
Version Control	Git / GitHub

Appendix C – Experimental / Raw Data

The dataset structure includes the raw text, per-token language tags, detected slang tokens, normalized forms, and named-entity annotations, as illustrated in Figure 1. The actual raw annotated dataset should be retained separately or attached where institutional rules permit.

Appendix D – Ethics / Institutional Approval

If institutional ethics approval was required for use of social-media-derived text, it should be documented here. Otherwise, retain the responsible data-handling statement in Chapter 3.7.

Appendix E – Individual Contribution Evidence

Mandatory for team projects. Each student should attach evidence corresponding to their claimed contribution, such as version-control commits, annotation files, code modules, model checkpoints, evaluation outputs, test records, screenshots, or meeting records.

Student	Evidence Type	Evidence Description / File Reference
M.Rishitha(192424335)	Corpus annotation / normalization evidence	Annotation files, normalization lexicon/mapping, and validation records to be attached.

Student	Evidence Type	Evidence Description / File Reference
S.Padmaja(192424159)	Model / dashboard evidence	Fine-tuning notebooks, evaluation outputs, dashboard screenshots and testing records to be attached.
M.Manasa(192425189)	Model / NER evidence	NER annotation files, transformer fine-tuning notebooks, model checkpoints, evaluation outputs, and testing records to be attached.
Vimala.K(192424276)	Language identification / preprocessing evidence	Code-mixed dataset files, data-cleaning scripts, language identification module, preprocessing outputs, and validation records to be attached.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8) + Appendix E